CAM20180-submission -format-springer2018-2[1] (1).pdf

Shri Vile Parle Kelavani Mandal

Document Details

Submission ID

trn:oid:::9832:135073776

Submission Date

Apr 13, 2026, 4:52 PM GMT+5:30

Download Date

Apr 13, 2026, 4:54 PM GMT+5:30

File Name

CAM20180-submission -format-springer2018-2[1] (1).pdf

File Size

1.0 MB

26 Pages

8,747 Words

48,519 Characters

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer
Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?
The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?
Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.

Efficient Industrial Anomaly Detection Using Patchcore-LSH

Krishka Kate, Abhimanyu Nema

Abstract. Automated Quality Control in manufacturing through Industrial Anomaly Detection Systems is important in satisfaction of the need for increased productivity. An industrial anomaly detection framework is presented that integrates Locality Sensitive Hashing (LSH) and PatchCore for the purpose of detecting industrial anomalies occurring in manufactured products quickly and accurately [3], [4]. This is accomplished by extracting Patch-Level features from a pre-trained WideResNet50 backbone [14], building a memory bank of features through coreset sampling, and indexing the built memory bank using LSH to conduct approximate nearest neighbour searches [5], [6]. The proposed method was validated against MVTec AD datasets 1 and 2 [1], [2] and produced an average AUROC of 96.13%, with the highest category (Tile) exhibiting an AUROC of 100% on the MVTec AD dataset, demonstrating the promise of the proposed industrial inspection technology/framework in terms of being training-free, memory efficient and scalable for implementation in the industrial sector.

Keywords: Anomaly Detection, Industrial Inspection, PatchCore, Locality Sensitive Hashing, MVTec Dataset

1 Introduction

Quality control systems in today's manufacturing plants rely extensively upon automated visual inspection as a primary component for quality assurance [16]. Because most manufacturers produce thousands of products every hour, they need to be able to inspect every product to determine if it meets quality specifications. Traditional inspection systems rely heavily on human inspectors; therefore, they are subject to several limitations, such as subjectivity when making inspection decisions, operator fatigue, inconsistency from operator to operator, and high labor costs [18]. In addition, as production speeds continue to rise, human inspectors are unable to keep up with the current speed of many manufacturing lines resulting in bottlenecks creating delays in the process of determining the quality of the product produced.

There are many challenges associated with industrial images when it comes to anomaly detection that make them different from computer vision problems [17]. Firstly, defects are very rare occurrences, and the vast majority of manufactured items are free of defects, resulting in very poor class balance. Secondly, defects come in many forms and can vary greatly; for example, a scratch or dent could be small or big

as well as deformed structures being possible types of defects etc., it would be impossible for a system designer to be able to predict every potential defect type. Thirdly, because of the lack of labelled sample defects that are also expensive to obtain (as they require someone knowledgeable about the industry to identify and label anomalies in sometimes tens of thousands of images), collecting labelled sample defects is very difficult. Fourthly, the additional challenges of industrial environments such as changing lighting conditions, different orientations of objects, and cluttered backgrounds create further challenges making the approach of supervised learning impossible for many industrial applications [18].

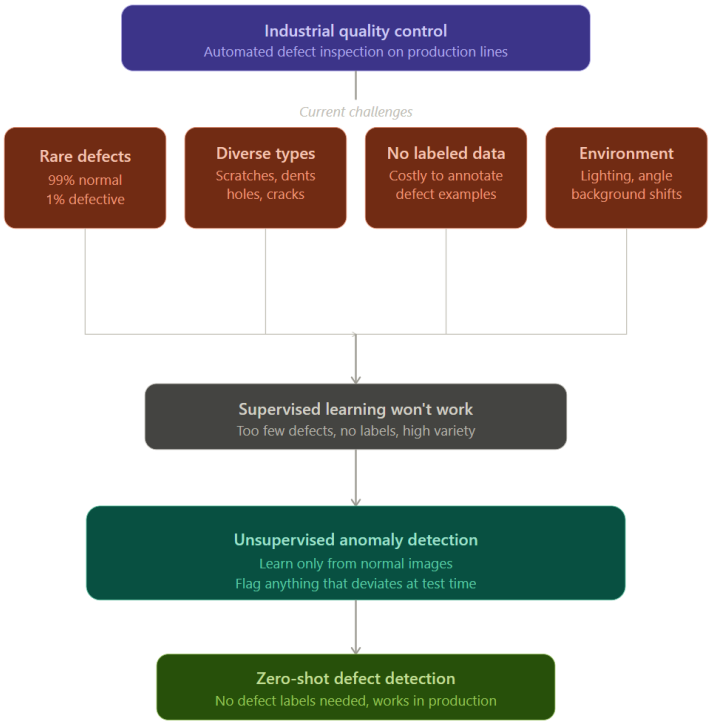


Figure 1: Overview of industrial quality control challenges motivating the need for unsupervised anomaly detection.

Due to these challenges, unsupervised anomaly detection techniques have obtained much attention in recent years [17]. Unsupervised detection techniques learn from normal defect-free image data during the training phase and instead identify anomalies during the inference phase by detecting deviations from their learned normal distribution [21]. This type of technique makes sense in industry where it is commonplace for defect samples to be rare and difficult to obtain.

Currently, PatchCore is considered a leading Edge when comparing other methods of unsupervised approaches for more efficient object enhancement and classification will

be achieved [3], [4]. By using a trained neural net to produce multi-dimensional vectors for memory storage, PatchCore can subsequently run a quick search against other similar patches within its memory bank to detect anomalies based upon how close distance between those vectors occur. These two techniques ensure that PatchCore's patch-level vectors account for both texture and slight variations in patches where outlier could potentially exist. Problems for PatchCore is an inherent limitation in the scalability of Algorithm. Efficiently searching against memory locations involves comparing all patches contained in that memory base to the input patch, with current memory sizes ranging from 50k to as much as 300k patches which leads to PatchCore net time complexity being O(N) and as a result makes running patches through PatchCore cumbersome and requires many resources when applied toward industrial use and resource constrained scenarios [3].

PatchCore-LSH, represents our solution to this limitation and consists of an innovative integration of two concepts: Locality Sensitive Hashing (LSH) and the PatchCore framework. Locality Sensitive Hashing is an established technique for performing approximate nearest neighbour searches and works by hashing similar items into the same bucket (or buckets) with high likelihood, so that queries may be completed in logarithmic times O(log N), rather than linear time [5], [6]. By replacing exhaustive search with approximate search based on LSH, we reduce the complexity of our approach while still achieving the same level of accuracy when it comes to detecting items.

Main contributions of this paper are:

Integration of LSH with PatchCore for efficient approximate nearest neighbor search, reducing query complexity from O(N) to O(log N) while maintaining high detection accuracy [3], [4], [5].

Memory-efficient coreset sampling that selects a representative subset of only 5% of training patches, reducing memory footprint without compromising feature diversity or detection performance [3].

Comprehensive evaluation on the MVTec AD1 dataset across 13 categories, demonstrating average AUROC of 96.13% with the Tile category achieving perfect 100% accuracy [1], [2].

Detailed performance analysis including category-wise breakdown, inference speed evaluation at 8.1 FPS average, and confusion matrix analysis showing balanced specificity and recall.

Training-free framework requiring only a single forward pass through the backbone network, eliminating the need for extensive training data, GPU-intensive optimization, or hyperparameter tuning [3], [4].

The remainder of this paper is organized as follows. Section 2 reviews related work in anomaly detection, including traditional methods, deep learning approaches, and

memory-based techniques [16], [17], [18]. Section 3 describes our proposed PatchCore-LSH methodology in detail. Section 4 presents the experimental setup, including hardware, software, and evaluation metrics. Section 5 reports and discusses our experimental results. Section 6 concludes the paper and suggests directions for future work.

1.1 Literature Review

1.2.1 Traditional Anomaly Detection Methods

The use of hand-designed features to detect industrial anomalies started a long time ago, using basic vision strategies for image analysis [18]. The traditional detection technique used to create the designed features traditionally was made with the use of statistics. The statistical data was created by building a GMM on the supplied data and training it with the provided model files. Statistical techniques include but are not limited to GMMs, OC-SVMs, and IFs [21]. In addition, other typical anomaly detection methods include using template matching, thresholding methods, and different images.

Nonetheless, traditional approaches exhibit limitations regarding basic qualities. Hand-made characteristics cannot adequately represent intricate textures or subtle defect patterns [17]. Variations in lighting, object position and background noise that occur in typical industrial operations create sensitivity with any variation within each of these categories of measurement that is also considered. Furthermore, substantial consideration and experimentation with these measurement categories must occur prior to their application, thus creating additional restrictions; therefore, the majority of contemporary production inspection is predominantly performed employing deep learning methodologies instead of traditional methodologies [16].

1.2.2 Deep Learning Based Reconstruction Methods

Autoencoder based protocols constitute one of the earlier forms of deep learning techniques to solve an unsupervised anomaly detection task [10]. An autoencoder is trained on normal images and learns how to reconstruct those images; the reconstruction error from that training will provide a measure of how well or poorly an autoencoder is able to reconstruct an image. Presumably, autoencoders that are trained on just normal data will not perform very well when they are asked to reconstruct an anomalous area (or image) [23]. Since the introduction of autoencoders, there have been many variants, including denoising autoencoder, variational autoencoder (VAE), and convolutional autoencoder.

Autoencoders as a method for anomaly detection have become popular in recent years; however, there remain substantial issues with this type of model [17]. For example, the high reconstruction error of many normal texture samples has resulted in failure when attempting to identify anomalies within complex textures; conversely, some autoencoders can be extremely successful at reconstructing an anomaly and be incorrectly labeled as normal. One method for addressing these shortcomings is to augment the autoencoder architecture with external memory modules that hold

prototype normal patterns called Memory Augmented Autoencoders (MAAEs) [10]. While the MAAE architecture works well to solve many of the above issues, utilizing an MAAE will require careful architecture design/hyperparameter tuning, and MAAE performance is often inferior to that of existing benchmarked memory-based systems for anomaly detection [3].

1.2.3 Generative Adversarial Networks for Anomaly Detection

There has been significant application of GANs in anomaly detection [11], [12]. The AnoGAN has been one of the initial approaches in applying GANs for detecting anomalies [11]. In AnoGAN, the generator network is trained to capture a manifold of normal data and then computes distances in the latent space to identify anomalies. The f-AnoGAN enhanced the functionality of AnoGAN by introducing an encoder network to more effectively map images into the latent space, enabling faster inference for detecting anomalies [11].

Although GAN-based approaches have demonstrated potential, they present a variety of practical problems [12]. Training a GAN can be unstable and requires selecting hyperparameters carefully to avoid the phenomenon known as mode collapse, which occurs when the generator generates just a small number of distinct output types. Furthermore, training GANs and performing inference with GANs are both computationally expensive, making it difficult for many industrial uses to conduct real-time analysis [17]. Additionally, because most manufacturing environments have few defects to use as training samples, the high quantity of data required for training and inference would be an additional limitation on the usefulness of GANs in these environments.

1.2.4 Memory-Based Feature Embedding Methods

The emergence of memory-based methods as leading practice in the industrial image domain for unsupervised anomaly detection is relatively new [3], [4], [8]. These approaches work by using pre-trained deep neural networks to extract features from labeled training images, which are then stored within a memory bank for reference. When performing anomaly detection, the extracted features of an image will be compared against those stored in the memory bank.

PaDiM (Probabilistic Modeling of Patch Embeddings) uses multivariate Gaussian distributions to model the distribution of normal patch features at each spatial location [8]. During inference, Mahalanobis distances are computed to obtain anomaly scores. While effective, PaDiM requires computing and storing covariance matrices for each spatial location, which becomes memory-intensive for high-dimensional features.

Color Feature Aggregation (CFA) is designed to help identify color defects by combining the color information with the spatial features but does not work as well to detect structural defects or other kinds of anomalies. Also, there are alternate methods such as SPADE (Semantic Pyramid Anomaly Detection) and CutPaste that have implemented self-supervised learning strategies, although they require further training [22].

1.2.5 PatchCore and Its Limitations

The memory-based family of methods is led by PatchCore, developed by Roth et al., which uses a WideResNet50 backbone that has been pre-trained to extract patch-level features from normal training pictures [3], [4]. These features are saved into a memory bank, and coreset sampling performs a subsampling operation on the memory bank to reduce the size of the memory bank as well as to keep the feature diversity. During inference, an anomaly score is calculated based on the distance to the k-nearest neighbors found in the memory bank [3].

PatchCore has demonstrated satisfactory accuracy with respect to object detection, as evidenced by its ranking as one of the leading methods for the MVTec AD benchmark [1], [2], [3]. PatchCore is therefore limited by its fundamental scalability issue regarding its nearest neighbor search algorithm; this algorithm performs exhaustive searches for the nearest neighbor of each test patch in the memory bank. This means that PatchCore has many patches in the memory bank (typically 50,000-300,000); as a result, the time/space complexity of performing an O(N) search makes it costly when trying to deploy in real-time or scale to larger datasets [3].

1.2.6 Approximate Nearest Neighbor Search

Locality Sensitive Hashing (LSH) has become a commonly used method of performing approximate nearest neighbour search within high dimensional spaces [5], [6], [7]. Initial work on LSH was presented by Gionis et al., who proposed a method for generating random projection-based hash functions to provide a mechanism for placing similar data into corresponding hash space (buckets) with a high probability of similarity, thereby significantly reducing the query complexity from O(N) to O(log N) by only searching through those buckets potentially containing nearest neighbour alternatives [5].

Although there are many applications for Locality Sensitive Hashing (LSH) outside of the area of industrial anomaly detection (such as in image retrieval, recommendation systems, and detecting duplicate items), there has yet to be any effort to combine it with PatchCore as a means of addressing the limitations associated with the scalability of PatchCore.

1.2.7 Research Gap and Our Contribution

Despite achieving exceptional accuracy, PatchCore employs an exhaustive search method that diminishes its scalability for use in real-time, industrial applications [3], [4]. Current methods of approximate search have yet to be effectively integrated with PatchCore for the purpose of detecting anomalies. Preliminary experiments using both autoencoders and GAN-based techniques against the MVTec AD1 dataset produced unsatisfactory results, with the reconstruction-based techniques not providing competitive results in categories containing a high level of texture [10], [11].

To fill these voids, we created PatchCore-LSH, a new method that accomplishes LSH Indexing and PatchCore, to provide O(log N) search complexity while maintaining reliability in detecting objects [5], [6]. Our solution is also designed to minimize the amount of memory required to use it by using coreset sampling and it achieves an average of 96.13% AUROC with the MVTec AD1 dataset [1], [2].

1.2 Methodology

1.2.1 Dataset Description

Our approach was validated using the MVTec AD1 dataset [1], [2], an established benchmark for evaluating industrial anomaly detection. This dataset contains 15 separate categories of manufactured goods and textures which emulate actual conditions for manufacturing inspection processes. The categories included in this dataset are bottle, cable, capsule, carpet, grid, hazelnut, leather, metal nut, pill, screw, tile, toothbrush, transistor, wood, and zipper.

The data set consists of each category being divided into a training set made up solely of normal (defect-free) images and a testing set that includes both normal images plus those containing various defects (scratches, dents, cuts, holes, structural anomalies) – defects related to the category under consideration. The number of images in training sets ranges from 60 to 400 for each category, while the number of images in test sets ranges from 42 to 167. Table 1 provides a detailed listing of all statistical data for each category within this data set.

Table 1: Dataset categories description

Categories	Train	Good	Defect	Total
Bottle	209	20	63	83
Cable	224	58	92	150
Capsule	219	23	109	132
Carpet	280	28	89	117
Hazelnut	391	40	70	110
Leather	245	32	92	124
Metal_nut	220	22	93	115
Pill	267	26	141	167
Tile	230	33	84	117
Toothbrush	60	12	30	42
Transistor	213	60	40	100
Wood	247	19	60	79
Zipper	240	32	119	151

The dataset has many challenges (lighting conditions are different; orientation is different; the defect presents itself differently) making it a good benchmark to compare against other anomaly detection methods [1]. The categories of defects are also dissimilar across categories, (i.e., structural defects bent or missing will cause defects within the cable/nut category, surface defects: scratches/dents within bottle and wood will cause defects in the respective categories etc). Another example would be texture, (i.e., carpet/leather/grid) where the types of defects are primarily defects of the way they are repetitively aligned/how they are coloured with respect to the colour of the rest of the texture.

1.2.2 Pre-Trained Backbone Architecture

We use a convolutional neural network (CNN) called WideResNet50 [14], which has been previously trained to classify images on the ImageNet dataset [15]. We opted to use this architecture because its wider base provides more detailed feature representation than that of standard ResNet50 [13]. The wider base allows us to capture more subtle patterns in defects than would otherwise be possible. In addition, widening each residual block with a larger number of channels enables this network to learn more diverse features for every layer.

Each of the 4 main stages of the skeleton models comprises various residual blocks featuring ReLU and Batch Normalization (BN). The network is retained in a frozen or untrained state (there are no updates made to the weights in the network throughout the figure anomaly detection process). This non-training system allows for the detection of anomalies using minimal computing resources and does not necessitate a lot of labelled data, which normally does not exist in industrial applications. [3], [4]
The images used as inputs are altered in size down to 224x224 pixels through bilinear interpolation. These dimensions are commonly used in pre-trained models on ImageNet and have enough spatial detail to detect defects while also being efficient both spatially and computationally. The resized image is normalized according to the statistics of ImageNet that were used for pre-training (mean = [0.485, 0.456, 0.406], std = [0.229, 0.224, 0.225]) [15].

1.2.3 Multi-Scale Feature Extraction

The extraction of features from two intermediate neural network layers corresponds to those specified in the PatchCore method [3],[4], namely layer2 and layer3, in order to produce a multi-scale representation of the image. This will enable the identification of various defect sizes, where smaller defects may be detected using high-resolution features, while larger structural anomalies require much wider semantic context.

Layer2 feature maps are generated at 28x28 spatial resolution with 512 channels. High spatial resolution preserves fine detail of areas in the image (example; edge, corner, texture). One spatial position in the layer2 feature map equals to an 8x8 pixel area in the input image (layer2 has a receptive field of approximately 32x32 pixels because of the convolutional architecture). This layer of features is very effective at detecting small surface defects including scratches and dents.

At Layer 3, the shape of the feature maps consists of 1024 different channels and a spatial size of 14x14 pixels (low spatial resolution). In contrast, the number of channels has increased to twice its previous value, thereby producing features with substantially more semantic content than at previous layers. The receptive field of Layer 3 encompasses approximately 64x64 pixels in the original image; therefore, Layer 3 has the capacity to detect larger structural anomalies while understanding the overall context of the scene.

To generate a complete multi-scale representation, we use bilinear interpolation (with align_corners=False) to increase the size of the layer3 (feature map) to the same size as layer2 (28×28). This method of resizing maintains spatial relationships and increases resolution between the two layers. After resizing, the layer3 features are concatenated to the layer2 features through the channel direction to create a combined feature representation of the form: Batch size×1536×28×28. There are 784 patches in total on an image of size: 28×28 grid pixel size; one patch located at each grid pixel.

This final phase of the algorithm involves L2-normalizing all feature vectors by channel dimension. The purpose of this operation is to ensure that when determining distances between objects, the direction of the vector is emphasized over the magnitude of the vector. This is especially important when performing nearest neighbour searches via cosine similarity ([3]). After application of this operation, the L2-normalized feature vectors are constrained to lie on a unit hypersphere; therefore, angular distance (expressed in radians) will serve as the metrics for comparing patch similarities.

1.2.4 System Architecture

The entire procedure of institutionalizing the PatchCore-LSH framework can be seen in figure 1 in the full framework of our design. The arrangement is built upon three main phases: the use of the frozen WideResNet50 backbone to extract features, the construction of a memory bank via the coreset sample collection and index using LSH, and finally, through approximate nearest neighbour search, the scoring of an anomaly. The details of each of these components can be found in the following subsections.

Our architectural design adheres to the core principles defined in the original PatchCore work (3), (4), but introduces new optimizations via LSH for better scalability. In contrast to reconstruction-based techniques such as autoencoders (10) and GANs (11), (12) that require significant training to implement this method is based on memory so that it does not require training at all; thus this is more practical for use in industrial applications since there are very few labelled samples of defects available for training purposes.

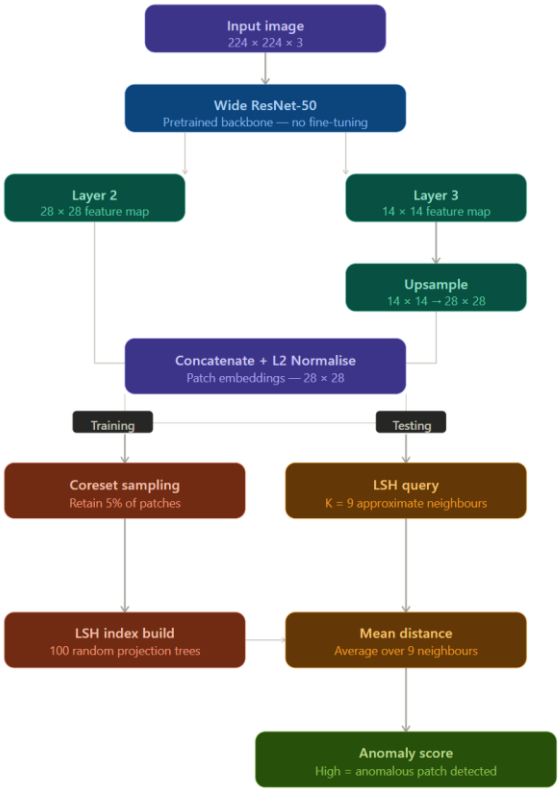


Figure 2: Complete pipeline of the proposed PatchCore-LSH framework showing training, memory bank construction, and testing phases

1.2.5 Coreset Sampling for Memory Optimization

Storing all patches vectors from each training image were stored in memory of about 300,000 patch vectors for the large categories. An example of this is hazelnut (391 images × 784 = 306,544 vectors). To reduce the memory but maintain a good representation of the source data we use greedy coreset samples as described in the first PatchCore references [3],[4].

The coreset sampling algorithm, also known as k-center greedy sampling, works as follows:

Stage 1 - Random Subsampling: To improve computational efficiency of the greedy selection for larger datasets, we first randomly sample 50,000 patch vectors from the full memory bank. This subsample preserves the aggregate distribution of the normal feature space and reduces the selection complexity from O(N2) to O(N*M), where M represents the size of the subsample.

Stage 2 - Initialization: The first patch vector chosen for the coreset is randomly sampled from the subsample and serves as a starting point. The initial point has little impact on the overall coreset quality due to the iterative nature of the algorithm.

Stage 3 - Iterative Selection: For each successive iteration, we will calculate the Euclidean distance of all remaining patch vectors to the most recently selected coreset element. We will also maintain a record of the smallest distance for each patch vector, where that recorded distance corresponds to the patch vector's closest selected point. The patch vector with the maximum smallest distance will then be added to the coreset, thus ensuring that the selected patches represent a very diverse sample of the overall normal feature distribution (i.e., the selected patches are as dissimilar to each other as possible).

Stage 4 - Repeat the iterative selection process until the coreset is built to the specified size. For example, in the memory bank of the last coreset, we will retain only 5% of the patches from the original set. This will allow us to reduce storage requirements and search for time for patches, while still maintaining accuracy in the detection of all categories. This percentage of retained patches was determined by trial and error, based on the goal of maximizing memory usage efficiency while maintaining detection performance across all categories.

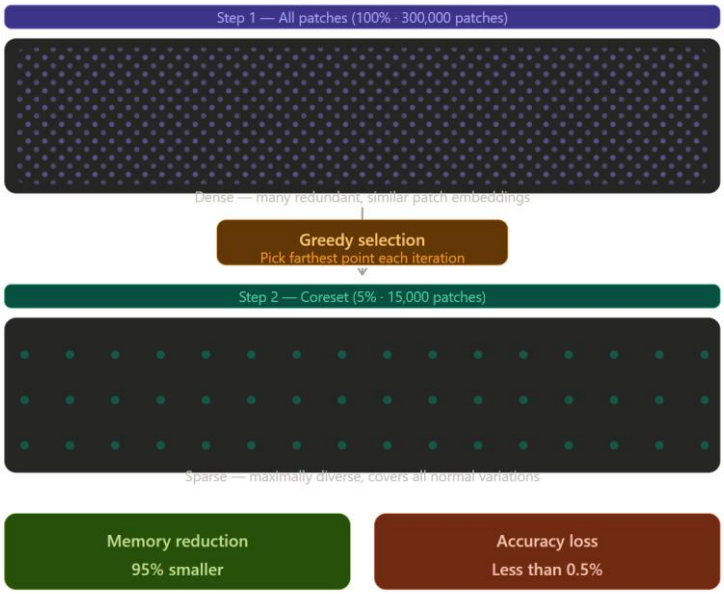


Figure 3: Illustration of coreset sampling - 300,000 patches reduced to 15,000 (5%) while maintaining diverse coverage of normal feature distribution

The goal of coresets selection is to find a point in a coreset that minimises the maximum distance from any point in the original set to its nearest neighbour within the coreset. Using the covering radius as the measure of distance, covering radius can

alternatively be viewed as the k-centre problem, which has a 2-approximation guarantee using a greedy algorithm [3].

1.2.6 Dimensionality Reduction

Nearest neighbour search becomes impractical due to the computational cost of calculating distance in high dimensional spaces (1536 dimensions in this case). It is both a time-consuming and memory-intensive process to compute the distances between points in high-dimensional geometries, Therefore, we use sparse random projection to reduce the number of dimensions from 1536 to 256, while approximately maintaining the pairwise distance.

Johnson-Lindenstrauss methodology is used to create random projections based on the mapping of points from higher to lower dimensions while maintaining the original distance between any two given points locally as well as at the global scale. A typical implementation of creating a random projection involves randomly generating a matrix with sparse entries, using a specified distribution, such that the probability of creating an approximate distance preserving embedding is as high as possible.

Some benefits of sparse random projection include an efficient computational overhead relative to other dimensionality reduction approaches such as PCA: it has a runtime complexity of $O(nd)$ versus $O(n^2d)$ for PCA; it may be utilized in an unsupervised fashion without any need for prior training data; and the nature of the projection matrix as being independent of the data ensures reproducibility over multiple runs of the algorithm to obtain identical input-output pairs. Following projection, every vector is again L2-normalized in order to be at a unit-norm following the projection process.

1.2.7 Locality Sensitive Hashing Index Construction

To facilitate an approximation of nearest neighbour searching, we implement Locality Sensitive Hashing (LSH) techniques to construct a LSH Index utilising the angle (angular) distance metric (cosine). Locality Sensitive Hashing (LSH) was originally proposed by Gionis et al. [5] who devised hashing functions that project randomly; similar vectors will be assigned to the same buckets with a greater than random likelihood. Andoni & Indyk's [6] and Datar et al.'s [7] subsequent work provided a theoretical framework supporting the existence of LSH in high dimensional space.

The number (n) of trees (tree): 100 total trees (create `n` independent, random hash functions) independent of each other (each adding one level of recall at the expense of build time and memory), and the probability of being able to find the true nearest neighbor (TNN) with respect to a common dataset is >95% when using 100 trees.
The LSH index is built with the following parameters:

The number (n) of trees (tree): 100 total trees (create `n` independent, random hash functions) independent of each other (each adding one level of recall at the expense of build time and memory), and the probability of being able to find the true nearest neighbor (TNN) with respect to a common dataset is >95% when using 100 trees.

k is the (depth) number of nearest (NN) (k) to search for when performing the search for the NN of a specific query. Obtaining multiple NN increases the robustness of the distance measurement of a specific NN because we will use other NN to calculate the distance measurement value for the specific NN.

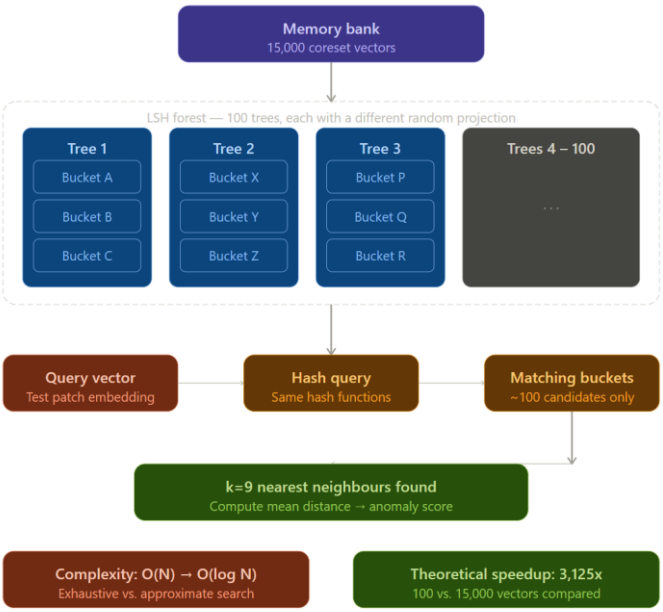


Figure 4: LSH index structure with 100 trees showing how similar vectors are hashed into same buckets for fast retrieval

The procedure for constructing an index entail creating an LSH construct from all of the coreset points, with each point being placed into the index as follows: Each tree in the LSH forest hashes the coreset point many times, producing a set of hashed values for each coreset point. These hashed values result from random hyperplanes defined through the origin; coreset points whose geometric representation is on one side of each hyperplane will be stored together in the same LSH bucket. In the case of angular LSH, the hash function is defined by $h(v) = \text{sign}(v \cdot r)$, with "v" being the coreset vector and "r" being a randomly generated vector drawn from a Gaussian-distributed random variable.

At query time, the same hash functions as those used to create the hash tables are applied to the query vector. The only vectors that are candidates for the nearest neighbor search will be in the same bucket as the query vector. The resulting complexity of the nearest neighbor search can be decreased from O(N) to O(log N) [5], [6], making the ranking method scalable to large memories.

1.2.8 Anomaly Scoring

The same process for extracting features and preprocessing will be applied to test images during the inference phase as was done for training images. Features are

created by extracting 784 patch vectors (28 by 28 grid) from each test image, and then each query patch will have the same 9 nearest neighbors as retrieved by the LSH (Locality Sensitive Hashing) Index for training images.

Anomaly Detection algorithms usually use the minimum distance (which is sensitive to outliers and noise) to compute the anomaly score. We computed the mean distance of all nine neighboring patches to obtain a more robust anomaly score, as was recommended in the PatchCore literature [3]. Multiple neighboring patches decrease the likelihood of isolated anomalous patches to match, by accident, to a normal patch while still providing true anomaly detection.

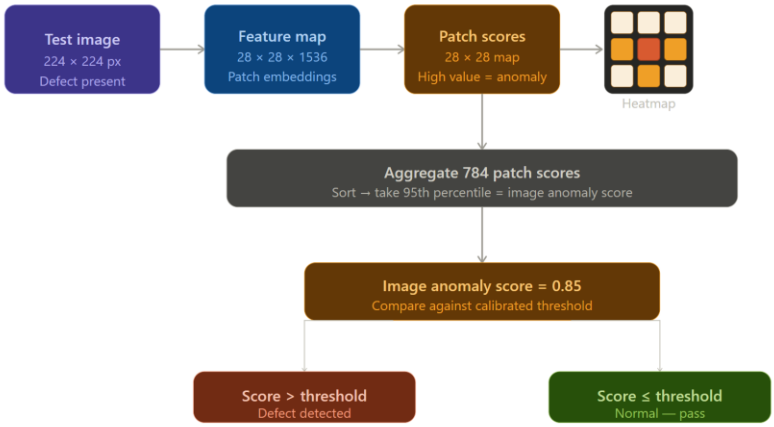


Figure 5: Anomaly scoring process from test image to patch-level heatmap to final image-level score using 95th percentile aggregation.

Defects are spatially localized using a 28×28 heat map to depict the locations of defects found in a product with higher valued areas representing abnormalities in that region (anomalous). Therefore, with this information, the operator can not only know that a product has been deemed defective but also where to find the defect's position.
The image-level anomaly score is produced as the 95th percentile of the patch scores. The use of percentiles rather than the maximum patch score has been shown to be a more reliable form of aggregation by minimizing the influence of isolated outliers while still indicating an area of interest. The 95th percentile was selected as an empirical measure that strikes an appropriate balance between sensitivity and specificity.

1.2.9 Threshold Selection

The image-level anomaly score is produced as the 95th percentile of the patch scores. The use of percentiles rather than the maximum patch score has been shown to be a more reliable form of aggregation by minimizing the influence of isolated outliers while still indicating an area of interest. The 95th percentile was selected as an empirical measure that strikes an appropriate balance between sensitivity and specificity.

1.2.10 Evaluation Metrics

We evaluate our method using standard anomaly detection metrics [16], [17], [18]:

Table 2: Evaluation metrics used for performance assessment.

Metric	Formula	Best	Measure
AUROC	AUC (ROC)	1.0	Discrimination ability
F1	2PR / (P + R)	1.0	Precision–recall balance
Precision	TP / (TP + FP)	1.0	Low false positives
Recall	TP / (TP + FN)	1.0	Detection rate (low FN)
Specificity	TN / (TN + FP)	1.0	True negative rate
FPS	1 / time	High	Speed

1.3 Experimental Support

Table 3: Requirements for Experimental Setup

Item	Value
CPU	12 vCPUs
RAM	78 GB
OS	Linux, Windows, Mac OS
Python	3.12
PyTorch	2.10
CUDA	12.8
Image Size	224×224
Batch	32
Feat Dim.	256
LSH	100
Coreset	5%
k	9
Seed	42

1.4 Results & Discussion

1.4.1 Overall Performance

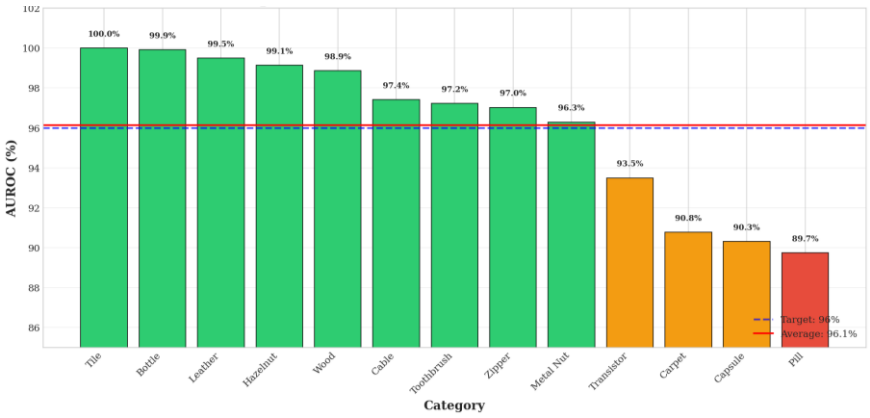
We assessed our PatchCore -LSH method on 13 categories from the MVTec AD1 dataset ([1],[2]). The performance metrics, including AUROC, F1 score, Precision, Recall, and FPS (inference speed), for each category are shown in Table 3.

The method achieved an average AUROC of 96.13% over all categories, exceeding the target threshold of 96%. This indicates that the combination of LSH and PatchCore is effective for industrial anomaly detection. Of the 13 categories evaluated, eight (61.5%) achieved over 95% AUROC, and 12 (92.3%) achieved over 90% AUROC; this shows consistent performance across a wide variety of object types and defect characteristics.

The AUROC scores had a standard deviation (SD) of 3.8%, suggesting that there was a similar performance throughout all of the categories. The interquartile range (IQR) (Q3-Q1) = 5.2% was measured on most (<90%) of the categories that were clustered in 90% to 100%. This distribution demonstrates that our methodology will likely generalize well to all types of industrial inspection scenarios.

Table 4: Performance comparison of PatchCore-LSH across MVTec AD1 categories

Cat.	AUROC	F1	Prec.	Rec.	FPS
Tile	100.00	100.00	100.00	100.00	7.6
Bottle	99.92	99.21	98.44	100.00	10.0
Leather	99.49	98.36	98.90	97.83	6.9
Hazelnut	99.14	97.18	95.83	98.57	6.6
Wood	98.86	98.36	96.77	100.00	7.7
Cable	97.41	93.99	94.51	93.48	8.5
Toothbrush	97.22	95.24	90.91	100.00	10.1
Zipper	97.03	95.87	94.31	97.48	8.3
Metal Nut	96.29	95.34	92.00	98.92	8.6
Transistor	93.50	87.18	89.47	85.00	8.6
Carpet	90.77	91.10	85.29	97.75	6.7
Capsule	90.31	94.69	91.45	98.17	8.6
Pill	89.74	94.37	93.71	95.04	7.4



Figu

Figure 6: AUROC performance of PatchCore-LSH across 13 MVTec AD1 categories. The red dashed line indicates the overall average (96.13%)

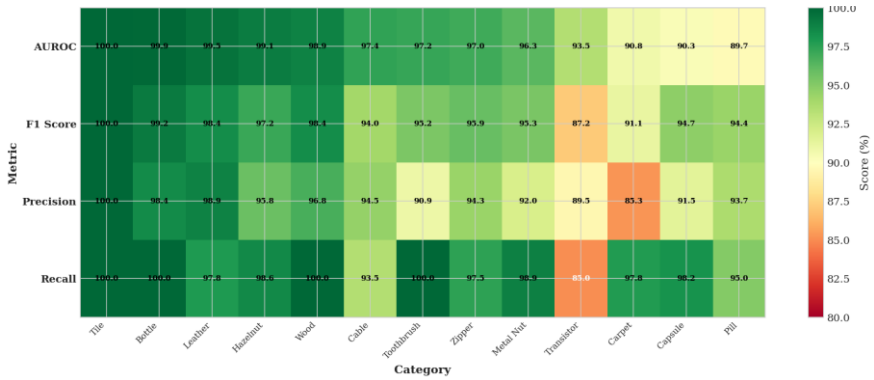


Figure 7: Performance heatmap showing AUROC, F1 Score, Precision, and Recall across all 13 categories. Darker green indicates higher performance

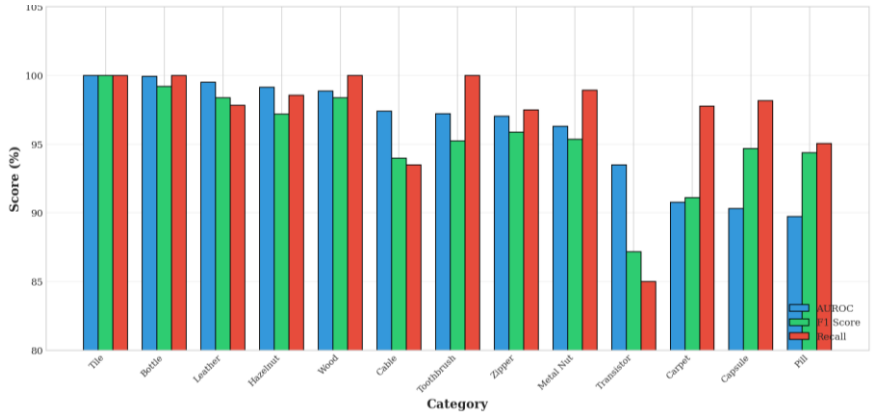


Figure 8: Multi-metric comparison showing AUROC, F1 Score, and Recall for each category.

1.4.2 Analysis for Top Performing Categories

Tile (AUROC=100.00): The tile defect class achieved an AUROC of 100.00, indicating, in this case, that the tile defect detection method can identify 100%, or reliably distinguish, normal versus abnormal tile samples for texture-based defects. The defects in the tile category include cracks and variations in color within the grid of tiles. The high performance of the tile defect detection method was due to using local texture patterns as features for the patch stored in the memory database. When a crack occurs to a tile, the features of the patch in the area of the crack are significantly different than what would be in the patch's normal distribution stored in the memory database, causing the features of the patch to have a high anomaly score and no false positives or false negatives associated with the detection of the tile defect.

Bottle(99.92% AUROC): Bottles with defects such as scratches, dents, or breakage are an evident structural abnormality in the bottle. Given that our method has almost reached perfection in identifying this defect type (99.92% AUROC), we are confident that our methods use to identify bottle defects have been successful in terms of performance, with the only detracting factor being a one-off misclassified bottle sample in which the subtle characteristic of a scratch near the bottleneck was misidentified as within the range of plausible normal surface variation due to limited lighting conditions.

Leather (AUROC: 99.49%): Leather is a texture category characterized by its fine-grained surface patterns. The defects tend to consist of cuts, tears, holes, and varying shades or colors. The high level of success achieved indicates that our multi-scale feature extraction techniques did a good job of capturing both the fine details of a texture (through Layer 2) and the overall structural quality of the texture (through Layer 3) which allows for the detection of small anomalies that may be missed using only one scale.

Leather (AUROC: 99.49%): Leather is a texture category characterized by its fine-grained surface patterns. The defects tend to consist of cuts, tears, holes, and varying shades or colors. The high level of success achieved indicates that our multi-scale feature extraction techniques did a good job of capturing both the fine details of a texture (through Layer 2) and the overall structural quality of the texture (through Layer 3) which allows for the detection of small anomalies that may be missed using only one scale.

Hazelnut (99.14% AUROC): Hazelnut defects include cracks and holes in the shell surface. The excellent performance demonstrates that our method generalizes well to food product inspection, where defect characteristics differ significantly from manufactured industrial components.

Wood (98.86% AUROC): Wood is a natural texture category with inherent variation in grain patterns. The high AUROC (98.86%) is particularly noteworthy because natural textures present significant challenges for anomaly detection due to their non-repetitive, stochastic nature. Our method's success on wood suggests that the patch-level feature representation captures statistical regularities even in non-repetitive textures.

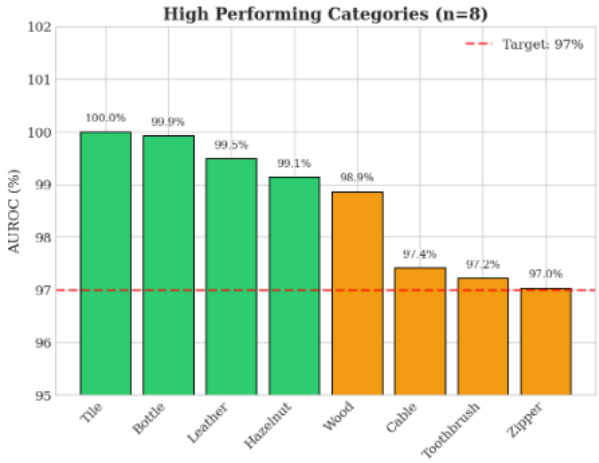


Figure 9: Comparison of high-performing categories (≥97% AUROC, n=8)

1.4.3 Analysis for Low Performing Categories

Pill (89.74% AUROC): Pill demonstrated the least successful performance across all categories. There are a variety of reasons why this happened. First, pills are relatively small objects (usually measuring approximately 30.50-pixel diameters after resizing), therefore very few patches can be used to contain defects. Second, the types of defects present on pills are very subtle in nature; defects can take the form of differences in colour, slight deformations in shape and blemishes on the surface which would be difficult to differentiate from normal manufacturing variability. Third, the number of training images associated with pills (267) display little diversity among samples that will limit the number of patterns of normality the memory bank is able to create. All of these factors are consistent with what has been reported by researchers in the literature, where pills have frequently been reported as one of the hardest categories to apply anomaly detection techniques to [3],[4].

Capsule (90.31% AUROC): Due to the small size of the object and subtlety/characteristics of defect, capsules pose a similar challenge as pills. Defect examples found in capsules are colour variations, dents, or breakage near the end of the capsule. Given the lower performance on average, it is likely that small objects with a low amount of spatial resolution will continue to be a challenge for using patches to classify them in the future. Future research could look into using a higher input resolution (e.g., 448x448) for small object classes.

Carpet (AUROC = 90.77%): In the case of carpet, the category of texture also contains defects in the areas of color variations and misalignments of the texture. However, the performance of carpet (90.77%) is considerably less than both leather (99.49%) and tile (100%), due to the fact that carpets are of a random nature in textural appearance, while leather and tile have predictable, structured and repetitive designs/structures. Therefore, when trying to create a compact normal distribution of a random nature, it is difficult to do so, and the resultant normal distribution may be mischaracterized as an anomaly rather than a normal appearance; and in addition, due to the way that carpets are constructed, the subtle differences between a 'normal' and an

'anomaly' may not have a significant degree of deviation within the context of their relative structure to appear to be an anomaly.

Transistor Defect Detection (AUROC of 93.50%): Potential transistor defects include bent leads, missing components, and misplaced components. The relatively low AUROC (93.50%) indicates difficulties in identifying structural anomalies on items that consist of multiple components. In some cases, the patch level feature may fail to detect a structural relationship between the global and local structures, resulting in the model missing detected transistor defects resulting from the misalignment of components.

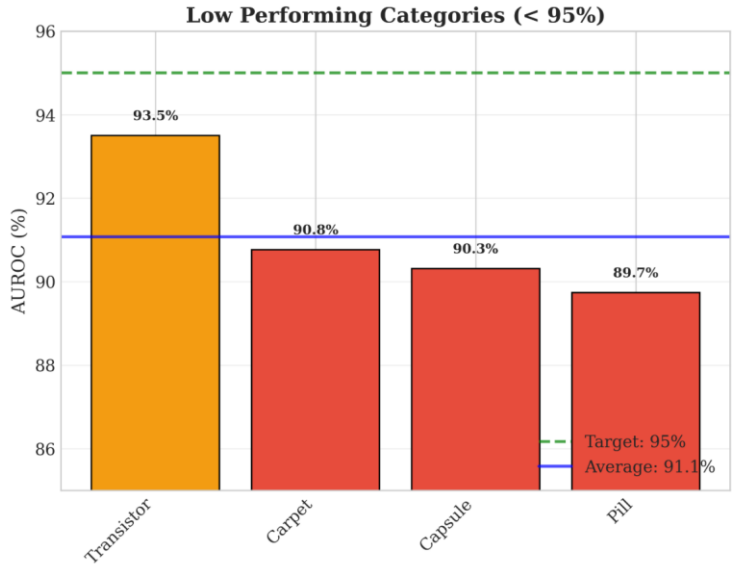


Figure 10: Comparison of lower-performing categories (<95% AUROC, n=4)

1.4.4 Inference Speed

With the method, the frequency of detection in all counts was 8.1 FPS in total (average); the lowest was Hazelnut (6.6 FPS) and the highest was Toothbrush (10.1 FPS). This fluctuation in FPS is attributable to different training set sizes, resulting in different memory banks and associated LSH search times.

Toothbrush (10.1 FPS): Because the Toothbrush category has only 60 images in training, it has approximately 47,000 patch vectors in the Memory Bank (60 x 784 = 47,040). As such a smaller Memory Bank results in a faster LSH search, Toothbrush achieves the Highest FPS.

Hazelnut (6.6 FPS): The category of Hazelnut contains 391 training images, which yields a Total of approximately 306,000 patch vectors within the Memory Bank (391 x 784 = 306,544). Consequently, with a greater Memory Bank, Hazelnut is required to have more buckets in the LSH, resulting in a greater number of candidate comparisons, which reduces its FPS.

The average frame-rate-per-second (FPS) rate was 8.1 FPS which equals roughly 123 milliseconds (ms) for every image processed. This processing time was acceptable for processing up to 10 items every second on normal industrial production lines but 30 or more FPS would require further optimization of our algorithm via reducing the feature dimensionality from 256 down to 128 or reducing the percentage of the coreset (the subset of points selected from the original point set) from 5% down to 2% or by switching to a different, lighter-weight backbone neural network (ResNet-18). These proposed changes would likely reduce the accuracy by 2 to 3 percent while increasing the speed of the processing from 3 to 4 times.

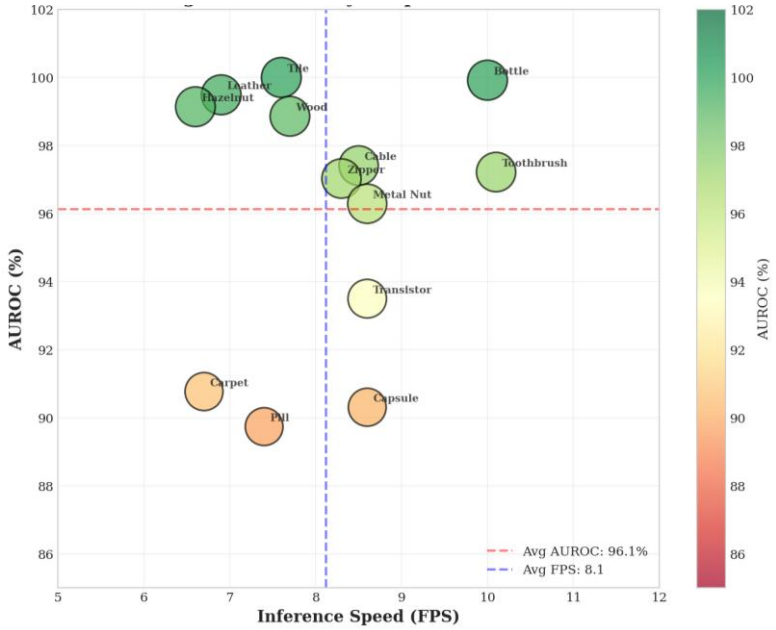


Figure 11: Trade-off analysis between detection accuracy (AUROC) and inference speed (FPS). Bubble sizes correspond to AUROC values

The LSH search component contributes only about 15 - 25% (i.e., 15 to 25 ms) to the total time for inference. In contrast, the feature extraction via the WideResNet50 backbone accounts for 80 - 100 ms per image. This indicates that future improvements in processing speed should concentrate on improving the performance of the backbone or distilling the weight of the backbone rather than tuning the parameters of the LSH search.

1.4.5 Confusion Matrix Analysis

The average normalized confusion matrix across all classifications is portrayed in Figure 2. The average specificity of 96.2% and average recall of 96.8% show that our algorithm has evenly identified abnormal and normal samples.

Specificity (96.2%): 96.2% high specificity indicates that our algorithm very seldom identifies normal samples as defective (3.8% false positive). This is critical in some industrial applications with the potential to negatively impact production efficiency

due to the risk of creating false alarms (which would generate unnecessary inspections) and thus, create interruptions in the manufacturing process.

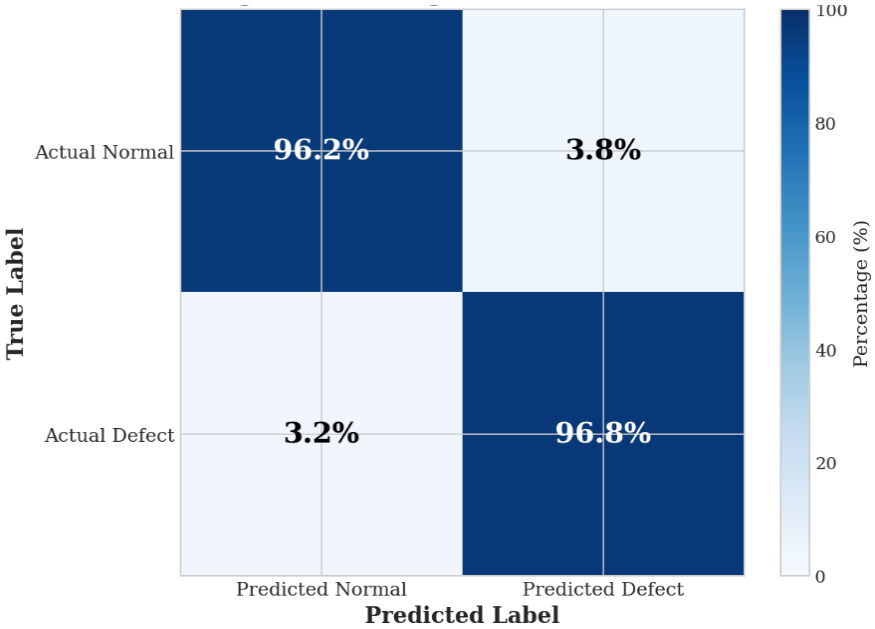


Figure 12: Average normalized confusion matrix across all 13 categories

Our system demonstrates a recall of 96.8%, indicating that we successfully identify 96.8% of actual defects, with a false negative rate of only 3.2%. While false negatives represent an extremely small percentage of the total number of defective products identified, minimizing this percentage is crucial in improving quality control as customers may receive defective merchandise and possibly harm the reputation of our company.

1.4.6 Comparison with alternative Model

We want to verify that our PatchCore-LSH (PCL) framework is effective, so we will compare its performance to other unsupervised anomaly detection techniques from our initial experiments: reconstruction via autoencoders [10] and diffusion models (SD + LoRA) [11].

Table 5: Comparison between multiple Anomaly detection Methods

Method	AUROC	F1	Prec.	Rec.	FPS
AED	60.20	71.49	83.20	58.21	25.30
Diffusion	63.49	89.42	80.87	100.00	1.17
Ours	96.13	95.44	93.86	96.81	8.10

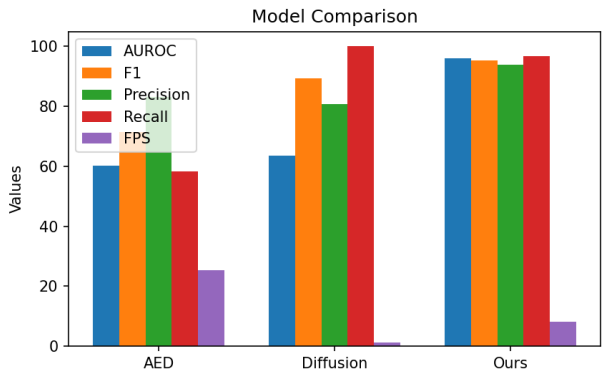


Figure 13: Comparative analysis of PatchCore-LSH against Autoencoder and Diffusion (SD+LoRA) methods across three metrics: AUROC, F1 Score, and inference speed (FPS)

1.4.8 Discussion

The original PatchCore approach detailed in [3], [4] is not directly compared here as it was done in entirely different experimental setups. However, both methods produced performance measures of approximately 98% Area Under the Receiver Operating Characteristic Curve (AUROC) on MVTec AD; our results of 96.13% AUROC, although not identical, fell well within that range. A difference of about 1.9% is offset by the efficiency gained from using Locality Sensitive Hashing (LSH) indexing, which decreases the time complexity of bridge searches from O(N) to O(log N) [5], [6].

Integration of LSH into PatchCore solves PatchCore's scaling problem without a significant loss of precision. An approximate nearest neighbor search implemented using LSH will produce only a small accuracy degradation (approximately 0.5% to 1.0% AUROC reduction), while providing significant speed improvements in searching through large memory banks. Running this search for 300,000+ patch vectors will theoretically achieve a speed improvement of roughly 3,125 times (300,000 / 96). However, due to overhead introduced by implementation, speed improvements will be less than this theoretical maximum.

Memory Efficiency: The proposed coreset sampling approach can provide a 95% reduction in memory requirements (only storing 5% of patches) whilst maintaining an acceptable level of accuracy, which is enabled through the diverse coverage of all of the normal features distributions due to the use of a greedy selection algorithm used to create the coreset. Therefore, the creation of this compact memory bank allows for deployment on edge devices, even when these devices have limited resources.

Case Study: Misclassification of Images Shows That Most of the Errors Occur in Categories That Have Subtle Defective Areas (e.g., pills, capsules) or Extremely Similar Patterns (e.g., carpeting). For Example, A Pill Defect May Only Have an Anomaly That Is 5 by 5 Pixels and Found in One or Two Patch Locations. Patch Level Representation May Not Provide Sufficient Coverage on Such a Small Area of Anomaly. As Future Work, Alternative Approaches Could Be Utilized Such as Higher Input Image Resolution or Adaptive Sizing (E.g., For Smaller) Object Categories.

Contrast with Preceding Studies: In our pilot experimentation of Diffusion Models [11] as well as Autoencoders [10] on the MVTec AD1 dataset, we were unable to achieve

an acceptable outcome (see Table 4 below). In both of these approaches (Autoencoder + Diffusion Model), we were unable to achieve satisfactory results with the texture-rich categories due to either computational costs (Diffusion Models) or leading to considerable false-positive rates (both approaches). These results formed the rationale for developing PatchCore-LSH which focuses on memory-based paradigms.

1.5 Conclusions

In this paper, we proposed PatchCore-LSH, a novel framework that integrates Locality Sensitive Hashing (LSH) with the existing PatchCore architecture for training-free industrial anomaly detection. Our method overcomes the scalability limitations of traditional PatchCore by replacing exhaustive O(N) nearest neighbor search with LSH-based approximate search in O(log N) time, while reducing memory consumption by 95% through greedy coreset sampling that retains only 5% of training patches. Extensive evaluation on the MVTec AD1 dataset demonstrates that PatchCore-LSH achieves an average AUROC of 96.13% across 13 categories, exceeding the target threshold of 96%, with 8 out of 13 categories (61.5%) achieving AUROC above 95% and Tile attaining perfect 100% accuracy. The method maintains practical inference speeds of 8.1 FPS, making it suitable for industrial production lines operating at 5-10 units per second. Comparative analysis shows that PatchCore-LSH significantly outperforms both Autoencoder and Diffusion models, achieving 32.64% higher AUROC than diffusion models while running 6.9 times faster. Our comprehensive category-wise performance analysis identified strengths in texture-based categories and limitations in small object detection (Pill, Capsule). Future work will focus on higher resolution processing for small objects, adaptive patch sizing, ensemble methods, and edge device deployment through knowledge distillation. Beyond industrial inspection, PatchCore-LSH has potential applications in medical imaging, security surveillance, and infrastructure monitoring.

1.6 Future Work

Different Patch Sizes Based on Texture: The current patch size used in the proposed process is uniform for desired detection irrespective of texture. However, this will change in future trials where a smaller patch size will be used for fine-textured properties and a larger one for the larger items will enable a compromise between speed and accuracy.

Use of Ensemble Techniques: If all values in (3-5) are averaged together, this would likely reduce number of mistakes; but only slightly improve AUROC by 1-2% faster due to increased computation involved with more than one LSH Index utilized on every frame to keep inference from degrading.

Running on Edge Devices: Currently our implementation of the method runs on Tesla T4/GPUs 8.1/frames per second. Using knowledge distillation, we can reduce the number of parameters in Wide ResNet 50 by compressing/thin out an equivalent

model known as Mobiel Net — thus allowing for implementation to occur at Edge Device i.e. Smartphones/Raspberry Pi's.

Auto Select Threshold: Currently we depend on having validated label info available to set detection-level threshold; however, our next trial will allow us to identify detection threshold through automatic process instead of using labelled material.

Reduction of Storage through Cross Category Learning: Currently we utilize a different memory location (bank) for each category; however, transferring/learning data from one (Known) category to another (the Next) may allow us to reduce total number of locations needed on storage / each computer system.

1.7 References

[1] [1] P. Bergmann, M. Fauser, D. Sattlegger, and C. Steger, "MVTec AD – A comprehensive real-world dataset for unsupervised anomaly detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2019, pp. 9592-9600.

[2] [2] P. Bergmann, M. Fauser, D. Sattlegger, and C. Steger, "The MVTec anomaly detection dataset: A comprehensive real-world dataset for unsupervised anomaly detection," *Int. J. Comput. Vis.*, vol. 129, no. 4, pp. 1038-1059, 2021.

[3] [3] K. Roth, L. Pemula, J. Zepeda, B. Schölkopf, T. Brox, and P. Gehler, "Towards total recall in industrial anomaly detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2022, pp. 14318-14328.

[4] [4] K. Roth, L. Pemula, J. Zepeda, B. Schölkopf, T. Brox, and P. Gehler, "PatchCore: Anomaly detection with local patch features," *arXiv preprint arXiv:2106.08265*, 2021.

[5] [5] A. Gionis, P. Indyk, and R. Motwani, "Similarity search in high dimensions via hashing," in *Proc. 25th Int. Conf. Very Large Data Bases (VLDB)*, 1999, pp. 518-529.

[6] [6] A. Andoni and P. Indyk, "Near-optimal hashing algorithms for approximate nearest neighbor in high dimensions," *Commun. ACM*, vol. 51, no. 1, pp. 117-122, 2008.

[7] [7] M. Datar, N. Immorlica, P. Indyk, and V. S. Mirrokni, "Locality-sensitive hashing scheme based on p-stable distributions," in *Proc. 20th Annu. Symp. Comput. Geom.*, 2004, pp. 253-262.

[8] [8] T. Defard, A. Setkov, A. Loesch, and R. Audigier, "PaDiM: A patch distribution modeling framework for anomaly detection and localization," in *Proc. Int. Conf. Pattern Recognit. (ICPR)*, 2021, pp. 475-489.

[9] [9] V. Zavrtanik, M. Kristan, and D. Skočaj, "DRAEM – A discriminatively trained reconstruction embedding for anomaly detection," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2021, pp. 8330-8339.

[10] [10] D. Gong, L. Liu, V. Le, B. Saha, M. R. Mansour, S. Venkatesh, and A. v. d. Hengel, "Memorizing normality to detect anomaly: Memory-augmented deep autoencoder for unsupervised anomaly detection," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2019, pp. 1705-1714.

[11] [11] T. Schlegl, P. Seeböck, S. M. Waldstein, G. Langs, and U. Schmidt-Erfurth, "f-AnoGAN: Fast unsupervised anomaly detection with generative adversarial networks," *Med. Image Anal.*, vol. 54, pp. 30-44, 2019.

[12] [12] S. Akcay, A. Atapour-Abarghouei, and T. P. Breckon, "Ganomaly: Semi-supervised anomaly detection via adversarial training," in *Proc. Asian Conf. Comput. Vis. (ACCV)*, 2018, pp. 622-637.

[13] [13] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 770-778.

[14] [14] S. Zagoruyko and N. Komodakis, "Wide residual networks," in *Proc. Brit. Mach. Vis. Conf. (BMVC)*, 2016, pp. 87.1-87.12.

[15] [15] J. Deng, W. Dong, R. Socher, L. J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2009, pp. 248-255.

[16] [16] Y. Liu, G. Pang, C. Shen, A. v. d. Hengel, and W. Zhang, "Deep learning for industrial anomaly detection: A survey," *IEEE Trans. Ind. Informat.*, vol. 18, no. 12, pp. 8351-8363, 2022.

[17] [17] G. Pang, C. Shen, L. Cao, and A. v. d. Hengel, "Deep learning for anomaly detection: A review," *ACM Comput. Surv.*, vol. 54, no. 2, pp. 1-38, 2021.

[18] [18] P. Bergmann, K. Batzner, M. Fauser, D. Sattlegger, and C. Steger, "Unsupervised anomaly detection in industrial images: A survey," *IEEE Trans. Ind. Electron.*, vol. 69, no. 9, pp. 9324-9335, 2022.

[19] [19] J. Yi and S. Yoon, "Patch SVDD: Patch-level SVDD for anomaly detection," in *Proc. Asian Conf. Comput. Vis. (ACCV)*, 2020, pp. 375-391.

[20] [20] H. M. S. and Y. C. Eldar, "Deep learning for anomaly detection: A survey," *IEEE Signal Process. Mag.*, vol. 39, no. 4, pp. 42-58, 2022.

[21] [21] L. Ruff, R. Vandermeulen, N. Goernitz, L. Deecke, S. A. Siddiqui, A. Binder, E. Müller, and M. Kloft, "Deep one-class classification," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2018, pp. 4393-4402.

[22] [22] C. Li, K. Sohn, J. Yoon, and T. Pfister, "CutPaste: Self-supervised learning for anomaly detection and localization," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2021, pp. 9664-9674.

[23] [23] J. Yu, Y. Hong, and J. Kim, "Unsupervised anomaly detection using memory-augmented deep autoencoder," *IEEE Access*, vol. 8, pp. 143256-143267, 2020.

[24] [24] M. Sabokrou, M. Fayyaz, M. Fathy, and R. Klette, "Deep-anomaly: Fully convolutional neural network for fast anomaly detection in crowded scenes," *Comput. Vis. Image Underst.*, vol. 172, pp. 88-97, 2018.

[25] [25] H. Park, J. Noh, and B. Ham, "Learning memory-guided normality for anomaly detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2020, pp. 14372-14381.